Tipos definidos por usuario, encapsulamineto

Programación II

06/0

8

Octubre

UNRN

Universidad Nacional de **Río Negro**

r27

Programación I

Tipos definidos por usuario I.

Universidad Nacional de Río Negro | Ingeniería en Sistemas | Ciclo 2026

Prof. Daniel Teira/Diego Diaz

Sede Bariloche

Contenidos de la clase

- Tipos Abstractos de Datos
- Encapsulamiento
- Implementación de .h y .c

Tipos Abstractos de Datos

1. Objetivo: diseñar componentes de software modulares, que se comunican con otros componentes y construyan una solución de software mayor y más compleja.
2. Vamos a empezar a ver algunas herramientas para encapsular funcionalidad pero el objetivo último es superior-
3. Veamos ejemplos de componentes compartidos en la industria automotriz .

Tipos Abstractos de Datos: ejemplos de modularidad





Tipos Abstractos de Datos





Tipos Abstractos de Datos

1. Como estos hay mil ejemplos…
2. ¿Para qué? ¿Por qué no hacer un motor para cada auto?

Tipos Abstractos de Datos

1. Puedo diseñar otros autos y REUTILIZAR el motor (stereo).

- Ya ganó confiabilidad. (Heritage).
- Ya conozco los pros y contras.
- Ya tengo desarrollada la cadena de repuestos.

2. Si saco un nuevo motor y no anda… cambio el motor pero no TODO el auto.

Tipos Abstractos de Datos

- En realidad el motor es un producto en sí mismo. Un tipo abstracto de datos es una unidad más chiquita.
  - Por ejemplo las bombas de agua, sistema de inyección, etc.
  -
- Pero una cuestión de escala/complejidad. El principio es el mismo.

Tipos Abstractos de Datos

- Un motor -o cualquier componente- tiene una función que cumplir, hace una tarea; y se comunica mediante interfaces:
  - Motor:
    - Los anclajes al chasis.
    - Conexión con la caja de cambios.
    - Conexión con el sistema de calefacción.
- Esto mismo vamos a aplicar a los componentes de software que hagamos.

Tipos Abstractos de Datos

1. ¿Por qué las abstracciones en programación?

- Las abstracciones reducen la complejidad al ocultar los detalles de implementación.
- Permiten enfocarse en qué debe hacer un sistema en lugar de perderse en cómo funciona exactamente por dentro.
- Fomentan la reutilización de código y el diseño modular mediante interfaces claras y componentes desacoplados.
- Facilitan el mantenimiento y la evolución del software, ya que los cambios internos no afectan al resto del sistema.
- Gracias a ellas, es posible construir sistemas complejos de gran escala de forma organizada, comprensible y colaborativa.

Tipos Abstractos de Datos

1. **Abstracción:**
   nos permite concentrarnos en las propiedades de una estructura de datos, sin especificar los aspectos menos importantes.

2. **Tipo Abstracto de Datos:**
   una declaración de estructura de datos y un conjunto de operaciones que interviene la estructura. El usuario del TDA usa esas operaciones para *crear, manipular, consultar y destruir* ejemplares del tipo de datos.

Tipos Abstractos de Datos

1. Para definir un buen TAD debemos aplicar una serie de conceptos.

Tipos Abstractos de Datos

1. **Generalización:**
   Captar características y comportamientos comunes de problemas o entidades específicas, para formular una solución más abstracta y reutilizable (ya sea un TDA genérico o un algoritmo universal).

2. **Encapsulamiento:**
   Agrupar la definición y el comportamiento del tipo de datos en una sola entidad y esconder los detalles de implementación

3. **Separación de la responsabilidad:**
   Responsabilidad en un módulo y no desparramada en varios lugares.

Tipos Abstractos de Datos

1. **Ocultamiento de la información (encapsulamiento):**
   La idea clave es el *ocultamiento de la información*. El TDA mantiene escondidos los datos internos, y solo son accesibles mediante operaciones definidas en su *interfaz.*

   Esta idea permite escalar (crecer) las soluciones y asegurarse de que encajen al integrarse.

Componentes de un TAD

1. Los componentes mínimos de un TDA pertenecen a tres categorías:
   a. *Estado interno:* variables que guardan información.
   b. *Constructores: funciones para crear* una instancia.
   c. *Funciones de acceso:* devuelven información de la instancia. NO lo modifican.
   d. *Procedimientos de manipulación: modifican el objeto.*
   e. *Destructores: devuelven los recursos utilizados y destruyen la instancia.*

2. Por otro lado podemos construir bibliotecas de funciones. del TAD. La regla es que no deberían acceder directamente a la implementación sino que deben usar las funciones del TAD.

Tipos Abstractos de Datos

Metodología:

1) Capturar el comportamiento deseado.
2) Definir el tipo de datos (su estructura interna).
3) Definir las operaciones del tipo de datos.
4) Implementación:
    a) TDA: utilizando herramientas de abstracción del lenguaje.
    b) Operaciones: con las estructuras de control del lenguaje.

Tipos Abstractos de Datos en C

Vamos a ver las herramientas de C para implementar TADs y resolver:

- Generalización.
- Encapsulamiento.
- Separación de responsabilidades.
- Ocultamiento de la información.

01 Tipo de datos definidos por usuario (Estructuras)

- Para definir TADs en C vamos a usar estructuras (struct) para agrupar información.

1

```
struct Persona {
    char nombre[50];
    int edad;
    float altura;
};

// Uso:
struct Persona empleado1 = {"Carlos", 30, 1.75};
```

Una estructura agrupa múltiples variables de **diferentes tipos de datos** bajo un mismo nombre. Cada variable dentro de la estructura se denomina *"miembro" o "campo" o “atributo”*.

01 Tipo de datos definidos por usuario (Estructuras)

- En memoria, los atributos de las estructuras se posicionan de forma secuencia, uno detrás del otro. El compilador alinea los atributos de acuerdo a la arquitectura.

1

```
struct Ejemplo {
    char att1; // 1 byte
    // El compilador añade 3 bytes de relleno aquí
    int att2; // 4 bytes
    float att3;
    };
```

Byte	Valor			
0x0	1 byte (att1	PAD	PAD	PAD
0x4	4 bytes (att2)			

- *Aunque 1 + 4 = 5, el tamaño real en memoria (`sizeof`) será de **8 bytes** debido a la alineación.*

01 Repaso de struct: acceso a los miembros

1

```c
#include <stdio.h>

struct {
    int codigo;
    float precio;
    char categoria;
} Producto;
```

```c
int main() { //inicialización
    Producto prod1 = {101, 249.99, 'A'};  //Inicializacion
    Producto prod2;
    Producto* prod3;

   //en el futuro vamos a evitar acceder a los atributos de la estrctura
fuera de la definición de la estructura.
    prod2.codigo = 102; //"." para asignar
    prod2.precio = 15.50;
    prod2.categoria = 'B';

    printf("--- PRODUCTO 1 ---\n");
    printf("Código: %d\n", prod1 .codigo); // "." para acceder

    printf("Precio: $%.2f\n", prod1.precio);
    printf("Categoría de envío: %c\n", prod1.categoria);

    prod3 = &prod2;       //obtengo un puntero a prod2.
    printf("\n--- PRODUCTO 2 ---\n");
    printf("Código: %d\n", prod3 ->codigo); // "->" si es puntero
    printf("Precio: $%.2f\n", prod2.precio);
    printf("Categoría de envío: %c\n", prod2.categoria);
    return 0;
}
```

1 01 Estructuras en memoria

```c
#include <stdio.h>

typedef struct Producto{
    int codigo;        // 4 bytes
    float precio;      // 4 bytes
    char categoria;    // 1 byte
} Producto_t;

int main() {

    //Análisis de memoria
    printf("\n--- Detalle de Memoria ---\n");
    printf("Tamaño teórico de los miembros: 4 + 4 + 1 = 9 bytes\n");
    printf("Tamaño REAL en memoria (sizeof): %zu bytes\n", sizeof(Producto));

    return 0;
}
```

Si sumas los bytes de cada miembro (int de 4 bytes + float de 4 bytes + char de 1 byte), el total teórico es 9 bytes. Sin embargo, cuando ejecutes este código, el sizeof(Producto) te devolverá 12 bytes.



Es el Padding (relleno de memoria). El compilador añade 3 bytes fantasma justo después del char categoria para que el tamaño total de la estructura sea un múltiplo de 4. Esto le permite al procesador de tu computadora leer la estructura a máxima velocidad.

01 Tipo de datos definidos por usuario (Uniones).

2

```
union Dato {
    int entero;
    float decimal;
    char caracter;
};

// Uso:
union Dato miDato;
miDato.entero = 10; // Si ahora
asignas miDato.decimal, el entero se
sobrescribe.
```

Las uniones son similares a las estructuras en su sintaxis, pero todos sus miembros comparten la misma ubicación de memoria.

El tamaño de la unión es el tamaño de su miembro más grande.

Uso principal: ahorrar memoria cuando se sabe que solo uno de los miembros se utilizará a la vez.

Regla de memoria: El tamaño total de la unión equivale al **tamaño de su miembro más grande** (más el padding que requiera para alinearse).

01 Tipo de datos definidos por usuario (Uniones ejemplo).

```c
#include <stdio.h>

union DatoSensor {
    int unidades;        // 4 bytes
    float temperatura;   // 4 bytes
    char estado;         // 1 byte
};
```

```c
int main() {
    union DatoSensor miSensor;  //instancio

    printf("--- Uso de la Unión ---\n");

    miSensor.temperatura = 24.5; //guardo temperatura
    printf("Temperatura guardada: %.1f°C\n", miSensor.temperatura);

   //AHORA guardamos unidades (Esto va a SOBRESCRIBIR la
temperatura)
    miSensor.unidades = 150;
    printf("Unidades guardadas: %d\n", miSensor.unidades);

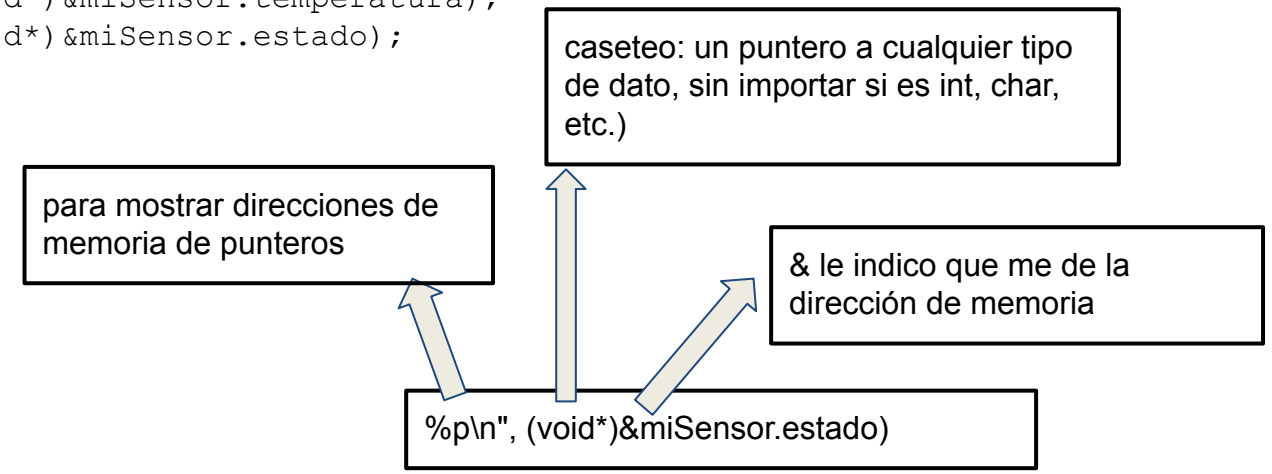
    //Que pasa si intentamos leer la temperatura ahora?
    // Como comparten la misma memoria, el valor viejo se corrompió
(se perdio)
    printf("Intento leer temperatura otra vez: %.1f°C (¡Dato
perdido!)\n", miSensor.temperatura);
```

2 01 Tipo de datos definidos por usuario (Uniones ejemplo).

```
int main()
{
    //Análisis de memoria crítico
    printf("\n--- Detalle de Memoria ---\n");
    printf("Tamaño del miembro más grande (int o float): 4 bytes\n");
    printf("Tamaño REAL de la unión (sizeof): %zu bytes\n", sizeof(DatoSensor));

    // Verificamos que todos los miembros apuntan a la misma dirección
    printf("Dirección de memoria de .unidades:    %p\n", (void*)&miSensor.unidades);
    printf("Dirección de memoria de .temperatura: %p\n", (void*)&miSensor.temperatura);
    printf("Dirección de memoria de .estado:      %p\n", (void*)&miSensor.estado);

    return 0;
}
```



01 Tipo de datos definidos por usuario (Enumeraciones).

3

```
enum DiasSemana {
    LUNES,     // 0
    MARTES,    // 1
    MIERCOLES // 2
};

// Uso:
enum DiasSemana diaHoy = LUNES;
```

Una enumeración consiste en un conjunto de **constantes integras con nombre**. Por defecto, al primer elemento se le asigna el valor `0`, al segundo `1`, y así sucesivamente, aunque se pueden forzar valores específicos.

- **Uso principal:** Para hacer que el código sea más legible reemplazando números por palabras descriptivas (por ejemplo, días de la semana, estados de un proceso).



Regla de memoria: Ocupa exactamente lo mismo que un tipo entero estándar de la arquitectura de tu máquina.

01 Tipo de datos definidos por usuario (Enumeraciones ejemplo I).

3

```c
#include <stdio.h>

enum EstadoSemaforo {
    ROJO,       // Internamente vale 0
    AMARILLO,   // Internamente vale 1
    VERDE       // Internamente vale 2
};

int main() {
    enum EstadoSemaforo estadoActual = VERDE; //Declaramos la variable a usar
    switch (estadoActual) { //Con case evaluamos
        case ROJO:
            printf("¡Alto! El semáforo está en ROJO.\n");
            break;
        case AMARILLO:
            printf("Precaución: El semáforo está en AMARILLO.\n");
            break;
        case VERDE:
            printf("Avance: El semáforo está en VERDE.\n");
            break;
    }
    //Como se ve en la memoria (como un entero)
    printf("El valor entero de 'estadoActual' es: %d\n", estadoActual);
    printf("El espacio que ocupa en memoria es de: %zu bytes\n", sizeof(estadoActual));
    return 0;
}
```

01 Tipo de datos definidos por usuario (typedef)

- En C existe una herramienta para generar un “alias” de una estructura de datos: typedef
- Uso principal: Para simplificar la sintaxis y evitar escribir la palabra struct, union o enum cada vez que se declara una variable.

```
struct Persona {
    char nombre[50];
    int edad;
    float altura;
};

// Uso:
struct Persona empleado1 =
{"Carlos", 30, 1.75};
```

```
typedef struct Persona {
    char nombre[50];
    int edad;
    float altura;
} Persona_t;

// Uso:
Persona_t empleado1 = {"Carlos",
30, 1.75};
```

01 Tipo de datos definidos por usuario (`typedef`)

- En C existe una herramienta para generar un “alias” de una estructura de datos: typedef
- Uso principal: Para simplificar la sintaxis y evitar escribir la palabra struct, union o enum cada vez que se declara una variable.

```
enum EstadoHTTP { //typedef y forzamos los
valores
    OK = 200,
    BAD_REQUEST = 400,
    UNAUTHORIZED = 401,
    NOT_FOUND = 404,
    INTERNAL_ERROR = 500
}; // 'EstadoHTTP' es ahora el alias directo

enum Estado HTTP stat;
```

```
typedef enum { //typedef y forzamos los valores
    OK = 200,
    BAD_REQUEST = 400,
    UNAUTHORIZED = 401,
    NOT_FOUND = 404,
    INTERNAL_ERROR = 500
} EstadoHTTP; // 'EstadoHTTP' es ahora el alias
directo

Estado HTTP stat;
```

Asignaciones, parámetros y retorno de función.

- Vimos struct, union y enum.
  - Enum se comporta como un int.
  - Si están compuestas de tipos básicos (int, char, float, etc), las estructuras se comportan según:
    - Al asignarse 2 estructuras, se copian elemento a elemento.
    - Si se pasan como parámetro por copia, se copian todos los elementos al stack.
    - Una función puede devolver una unión o una estructura copiando los valores de modo similar a como se pasan por copia.
  - ¿Qué pasa con la copia de estructuras si tienen punteros adentro?
    - ¡Tengo que implementar función de copia!

Asignaciones, parámetros y retorno de función.

- ¡Tengo que implementar función de copia!.....

```c
#include <stdio.h>
#include <stdlib.h>
// 1. Estructura interna (A la que apuntará la estructura principal)
typedef struct
{
int codigo;
float medicion;
} InformacionDetallada;

// 2. Estructura principal (Contiene un puntero interno)
typedef struct
{
int id; InformacionDetallada* detalle; // <- Puntero interno que requiere copia profunda
} Contenedor;
```

```c
// FUNCIÓN DE COPIA PROFUNDA (DEEP COPY)
Contenedor* clonar_contenedor(Contenedor* original)
{
if (original == NULL) return NULL;
// Paso 1: Reservar memoria para la nueva estructura principal (Copia) Contenedor*
copia = (Contenedor*) malloc(sizeof(Contenedor));
if (copia == NULL)
      return NULL;
// Paso 2: Copiar datos primitivos/básicos de la estructura principal
copia->id = original->id;
// Paso 3: Gestionar la copia del puntero interno de forma segura

if (original->detalle != NULL) {
// Reservamos un bloque de memoria NUEVO y exclusivo para el detalle de la copia
copia->detalle = (InformacionDetallada*) malloc(sizeof(InformacionDetallada));
if (copia->detalle == NULL)
      {
      free(copia); // Si falla, limpiamos lo anterior para evitar leaks
      return NULL;
      }
// Copiamos elemento a elemento los datos reales de la estructura interna
copia->detalle->codigo = original->detalle->codigo;
copia->detalle->medicion = original->detalle->medicion;
}
else
      {
      // Si el original no apuntaba a nada, la copia tampoco
      copia->detalle = NULL;
      }
return copia;
}
```

Resumen

Generalización:	Definimos un punto genérico. Puede ser un gráfico, una coordenada en un plano cualquiera o un punto en la pantalla del monitor.
Encapsulamiento	Los archivo h y c encapsulan toda la funcionalidad de Punto. No hay lógica que modifique un punto, dispersa en otros archivos.
Separación de la responsabilidad	La responsabilidad de operar con puntos quedará en el módulo.
Ocultamiento de la información.	El usuario de Punto no sabe como está implementado. Solo conoce el comportamiento.

Vamos a volver sobre esto cuando hagamos tipos de datos más complejos.

01 Tipo de datos definidos por usuario: Tipo Punto.

- Generar un tipo de datos Punto. Se trata de un punto en el plano cartesiano (X,Y).
- Debe tener funciones para sumar, restar, escalar y calcular módulo.

```
// Definición básica
struct PuntoTradicional {
    int x;
    int y;
};

// Para usarla en el main:
struct PuntoTradicional miPunto; // <-- Es obligatorio escribir "struct" adelante
miPunto.x = 10;
```

```
#include <stdio.h>

// Definimos la estructura y le asignamos el alias 'Coordenada' al final
typedef struct {
    int x;
    int y;
} Coordenada;

int main() {
    // ¡Mirá qué limpio! Ya NO hace falta escribir "struct Coordenada..."
    Coordenada puntoA = {5, 12};
    Coordenada puntoB;

    // Asignación clásica
    puntoB.x = 20;
    puntoB.y = 35;

    // Imprimir los datos
    printf("--- PUNTOS EN EL PLANO ---\n");
    printf("Punto A: (x: %d, y: %d)\n", puntoA.x, puntoA.y);
    printf("Punto B: (x: %d, y: %d)\n", puntoB.x, puntoB.y);
}
```

01 Tipo de datos definidos por usuario (typedef ejemplo II).

4

Código autodocumentado: Si ves `Moneda saldo;`, entendés al instante para qué sirve esa variable. Es mucho más claro que ver un simple `double saldo;`.

Fácil mantenimiento: Imaginá que el proyecto crece y ahora el saldo del cliente necesita más precisión o soporte para números masivos. En lugar de buscar y cambiar `double` por todo tu código (que podrían ser miles de líneas), solo cambiás la línea del `typedef` al principio del archivo:

```
#include <stdio.h>

// Se crea alias para tipos de datos primitivos
typedef unsigned long long NumeroCuenta;     // 'NumeroCuenta' es ahora un alias de un entero gigante sin signo
typedef double Moneda;                       // 'Moneda' es ahora un alias de un float de doble precisión

//También podemos usarlo para simplificar la creación de arreglos
typedef char Texto[50];                      // 'Texto' es ahora un alias para una cadena de 50 caracteres

int main() {
    //Declaramos variables usando nuestros nuevos tipos personalizados
    NumeroCuenta cuentaCliente = 4500123489761122ULL;
    Moneda saldoActual = 15250.75;
    Texto nombreCliente = "Juan Pérez";

    // 4. Mostramos los datos en pantalla
    printf("--- DATOS DEL CLIENTE ---\n");
    printf("Nombre: %s\n", nombreCliente);
    printf("Nro. de Cuenta: %llu\n", cuentaCliente);
    printf("Saldo: $%.2f\n", saldoActual);

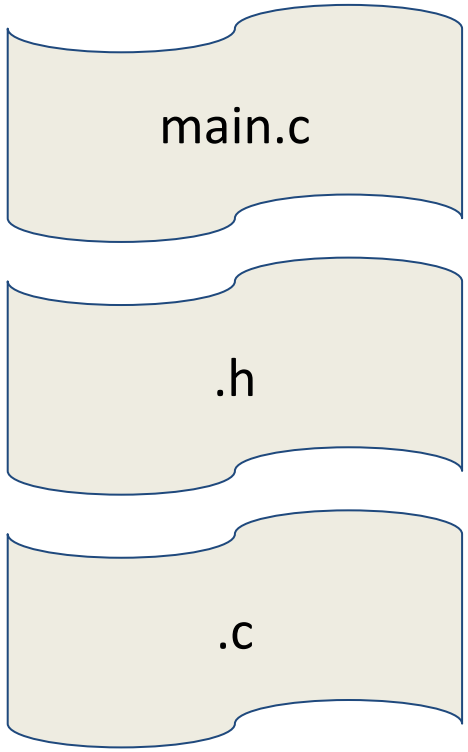
    // 5. Demostración de memoria
    printf("\n--- Detalle de Memoria ---\n");
    printf("Tamaño de 'Moneda' (double): %zu bytes\n", sizeof(Moneda));
    printf("Tamaño de 'Texto' (char[50]): %zu bytes\n", sizeof(Texto));

    return 0;
}
```

Implementación de .h y .c

Implementar archivos .h (cabecera) y .c (fuente) en C permite separar la interfaz (lo que hace un módulo) de la implementación (cómo lo hace), lo cual organiza y acelera el desarrollo de software

- **Modularidad:** Divide un programa grande en partes pequeñas y manejables. Cada módulo agrupa funciones y datos relacionados con un propósito específico.
- **Encapsulamiento:** El archivo .h funciona como un contrato o interfaz pública. Muestra qué funciones existen y cómo usarlas, mientras oculta los detalles internos de los archivos .c que pueden cambiar sin afectar al resto del programa.
- **Compilación por separado:** Permite compilar cada archivo .c de forma independiente en un archivo objeto (.o). Si modificas un solo archivo, solo se recompila ese archivo y no todo el proyecto, lo que ahorra mucho tiempo en proyectos grandes.
- **Reutilización de código:** Facilita usar las mismas funciones en diferentes proyectos con sólo copiar o incluir los archivos de cabecera.
- **Evita dependencias circulares y errores:** Permite que varios archivos fuente incluyan las declaraciones necesarias mediante `#include` sin duplicar código ni chocar con las definiciones reales que viven en el archivo `.c`



Implementación de .h y .c

El Archivo de Cabecera (`.h`): La Interfaz

Qué representa: Es el "manual de usuario" del código. Le dice al compilador qué herramientas están disponibles para ser usadas por otros archivos.

Qué debe incluir:

- **Prototipos de funciones:** Solo las firmas (tipo de retorno, nombre y argumentos). Ejemplo: `int calcular_trayectoria(int x, int y);`
- **Estructuras de datos y tipos:** Definiciones de `struct`, `enum` y `typedef` que el usuario del módulo necesite conocer.
- **Macros y Constantes:** Definiciones con `#define` que sean de uso público (ej. `#define PI 3.1415`).
- **Variables externas:** Declaraciones usando `extern` si un estado global debe compartirse (aunque se recomienda minimizar su uso).
- **Include Guards:** Las directivas `#ifndef`, `#define` y `#endif` para evitar que el archivo se incluya más de una vez durante la compilación, lo cual causaría errores.

Qué NO debe incluir:

- Nunca debe contener la lógica o el código interno de las funciones.

Implementación de .h y .c

El Archivo Fuente (.c): La Implementación

Qué representa: Es la caja negra. Aquí es donde los algoritmos cobran vida y donde la CPU gasta sus ciclos de reloj. Los otros archivos del programa no necesitan (ni deben) mirar aquí adentro.

Qué debe incluir:

- **La inclusión de su propio `.h`:** Lo primero que hace un `modulo.c` es un `#include "modulo.h"` para asegurarse de que sus implementaciones coincidan con el contrato prometido.
- **El código de las funciones:** El desarrollo paso a paso de los prototipos declarados en el `.h`.
- **Inclusiones de librerías privadas:** Librerías estándar (como `<math.h>` o `<string.h>`) que solo necesita este archivo para hacer sus cálculos internos.
- **Funciones e información privada:** Funciones auxiliares o variables globales marcadas con la palabra reservada `static`. Al usar `static`, esas funciones quedan "ocultas" al resto del programa y solo pueden ser llamadas desde dentro de este mismo archivo `.c`.

Implementación de .h y .c

Fases de la Construcción, como se compilan los archivos



1. Preprocesamiento (El Preparador)

Antes de compilar, el preprocesador lee el código y busca las líneas que empiezan con `#` (directivas).

- Cuando encuentra un `#include "modulo.h"`, literalmente **copia y pega** todo el contenido del archivo `.h` dentro del `.c` en ese exacto lugar.
- Reemplaza todas las macros definidas con `#define`.
- El compilador nunca ve un archivo `.h`; solo ve un archivo `.c` gigante y expandido.

2. Compilación y Ensamblado (El Traductor)

El compilador toma cada archivo `.c` (ya preprocesado) y lo traduce **de forma independiente** a lenguaje máquina.

- El compilador revisa la sintaxis. Si usaste una función de `modulo.h` en `main.c`, el compilador dice: *"No sé dónde está el código de esta función, pero confío en ti porque me mostraste su prototipo en el `.h`"*.
- El resultado de esta etapa son **archivos objeto** (con extensión `.o` en Linux/Mac o `.obj` en Windows). Estos archivos contienen código máquina, pero aún no se pueden ejecutar porque están desconectados.

Enlazado o Linking (El Ensamblador Final)

Aquí es donde entra el **Linker**. Su trabajo es tomar todos esos archivos objeto independientes y conectarlos entre sí.

- El Linker busca en `modulo.o` el código real de la función y conecta esa dirección de memoria con la llamada que hiciste desde `main.o`.
- También conecta tu código con librerías externas (como `printf` de la librería estándar de C).
- Si el Linker no encuentra la implementación de una función declarada en un `.h`, arrojará el famoso error de *Undefined reference*.
- El resultado final es el **archivo ejecutable** (ej. `.exe` o un archivo binario en Linux).

Implementación de .h y .c

Ejemplo muy básico para controlar la articulación de un brazo robótico.

El Archivo de Cabecera:

```
// brazo.h
#ifndef BRAZO_H
#define BRAZO_H

// Constante pública
#define ANGULO_MAXIMO 180
#define ANGULO_MINIMO 0

// Prototipo de la función pública
void mover_articulacion(int grados);

#endif // BRAZO_H
```

Archivo de Implementación:

```
#include <stdio.h>
#include "brazo.h"
static void enviar_senal_pwm(int grados)
{
printf("[Hardware] Generando señal PWM para %d grados...\n", grados);
}
void mover_articulacion(int grados)
{
   if (grados > ANGULO_MAXIMO || grados < ANGULO_MINIMO)
         {
         printf("Error: Movimiento fuera de rango de seguridad.\n");
         return;
         }
   printf("Comando recibido: Moviendo articulación a %d grados.\n", grados);
   enviar_senal_pwm(grados);
}
```

Implementación de .h y .c (El Archivo de Cabecera)

Ejemplo muy básico para controlar la articulación de un brazo robótico.

El Archivo de Cabecera:

```
// brazo.h
#ifndef BRAZO_H
#define BRAZO_H

// Constante pública
#define ANGULO_MAXIMO 180
#define ANGULO_MINIMO 0

// Prototipo de la función pública
void mover_articulacion(int grados);

#endif
```

#ifndef y ***#endif***: son directivas del preprocesador que sirven para controlar qué partes del código se deben compilar y cuáles no (compilación condicional)

Prototipo: de la función “publica”

Implementación de .h y .c (El Archivo de Implementación)

Ejemplo muy básico para controlar la articulación de un brazo robótico.

Static al usar static, convertimos a la función en privada, que significa que solo puede ser accedida por otras funciones en el archivo.

Esta función puede ser accedida desde main.

```
// brazo.c
#include <stdio.h>
#include "brazo.h"

static void enviar_senal_pwm(int grados)
        {
// Simulamos el envío de una señal de control a un servomotor
        printf("[Hardware] Generando señal PWM para %d grados...\n", grados);
        }

// Implementación de la función pública
        void mover_articulacion(int grados)
        {
        if (grados > ANGULO_MAXIMO || grados < ANGULO_MINIMO)
                {
                printf("Error: Movimiento fuera de rango de seguridad.\n");
                return;
                }
        printf("Comando recibido: Moviendo articulación a %d grados.\n", grados);
enviar_senal_pwm(grados);
        }
```

Implementación de .h y .c (El Programa Principal)

```
// main.c
#include <stdio.h>
#include "brazo.h"
// Incluimos nuestra interfaz
int main()
{
printf("Iniciando sistema de control...\n");
// Usamos la función disponible en la interfaz
mover_articulacion(45);
mover_articulacion(200); // Esto disparará la validación de
seguridad
return 0;
}
```

El archivo principal solo necesita incluir el .h (el manual) para saber cómo usar el brazo. No tiene idea de que internamente hay una función enviar_senal_pwm.

Implementación de .h y .c (Compilar)

Paso 1: Generar los archivos objeto de forma independiente (El Compilador)

```
gcc -c main.c     # Esto genera 'main.o'
gcc -c brazo.c    # Esto genera 'brazo.o'
```

Paso 2: Unir las piezas (El Linker)

```
gcc main.o brazo.o -o control_robot
```



Esto genera el archivo final ejecutable llamado `control_robot` (o `control_robot.exe` en Windows).

Ejecución:

```
./control_robot
```

Resultado

```
Iniciando sistema de control...
Comando recibido: Moviendo articulación a 45 grados.
[Hardware] Generando señal PWM para 45 grados...
Error: Movimiento fuera de rango de seguridad.
```

Ejercicio

El comedor universitario a entregar 500 viandas de almuerzo gratuitas por día. Al mediodía, se forma una fila enorme de estudiantes.

Para organizar la entrega y evitar abusos, hay una única regla de oro: **Cada estudiante puede retirar, como máximo, UNA vianda por día.**

El sistema funciona así:

1. Al abrir el comedor, el registro del día está en blanco.
2. Cuando un estudiante llega al mostrador, escanea su número de Legajo.
3. El sistema debe verificar instantáneamente si ese estudiante ya retiró su vianda hoy.
    - Si ya la retiró, suena una alarma de rechazo.
    - Si no la retiró, se le entrega la vianda y su Legajo queda registrado.
4. Si un estudiante retira la vianda pero al ver el menú decide devolverla intacta, su registro debe eliminarse para que pueda volver a intentarlo más tarde.
5. Al final del día, el centro de estudiantes debe tomar un Legajo al azar de los que comieron ese día para enviarle una encuesta de calidad del servicio.